AgentKillChain: A Prototype Benchmark for Persistent Compromise in Autonomous AI Agents

Author: Kevin O'Connor
 Version: Preprint candidate, May 20, 2026
 Repository: https://github.com/KevinOBytes/AgentKillChain

Abstract

Autonomous AI agents increasingly combine language models with persistent memory, retrieval, and tool execution. These capabilities expand the security boundary beyond a single model response: untrusted content can be summarized into memory, retrieved in later sessions, and used to influence tool selection. AgentKillChain is an open prototype benchmark for evaluating this class of persistent compromise risk. The benchmark models eight prompt-injection and memory-poisoning families across five workflow framings, then measures whether a simulated assistant loop produces unsafe instruction adoption, unsafe tool proposals, secret-disclosure signals, or explicit policy-violation markers. In a May 20, 2026 baseline over 39 authenticated Ollama Cloud models and 40 attack scenarios, the harness attempted 1,560 model-scenario pairs, scored 1,559, and observed a 1.48% overall unsafe-signal rate under hardened deterministic heuristic scoring. Unsafe signals were concentrated in single-phase toolchain-confusion and context-manipulation scenarios; the current latent-memory scenarios did not produce scored unsafe activation under this heuristic. These results do not prove broad real-world model compromise. They show that even a mocked, policy-gated agent loop can produce low-frequency unsafe behavior signals, supporting the need for memory provenance, deterministic tool authorization, and public benchmark methods that separate reproducible aggregate evidence from sensitive raw traces.

Keywords: AI security, autonomous agents, prompt injection, indirect prompt injection, memory poisoning, tool use, benchmark, Ollama

1. Introduction

Large language model applications are moving from stateless chat interfaces toward stateful agents that retrieve documents, summarize working context, maintain long-term memory, and call tools. This shift changes the threat model. A malicious instruction no longer needs to win in a single turn. It can be embedded in retrieved content, survive summarization, and reappear later when the agent is operating in a higher-risk workflow.

AgentKillChain evaluates this temporal risk. The benchmark asks whether a simulated assistant agent can preserve the boundary between instructions and untrusted data while handling poisoned documents, memory buffers, and tool-selection prompts. It is intentionally a prototype: tools are

mocked, memory is represented as a controlled context buffer, and scoring is conservative. The goal is not to claim a universal attack success rate. The goal is to create a repeatable test harness for a specific class of agent failure modes and to make the resulting aggregate evidence public without publishing private raw prompts or model outputs.

The May 20, 2026 baseline evaluates authenticated Ollama Cloud models using a hardened deterministic heuristic. It provides a useful first release because it completes the full 39-model catalog sweep, preserves run metadata, records coverage and error rates, and exposes the public result artifact in redacted form.

2. Contributions

This work contributes:

1. A prototype threat model for persistent compromise in autonomous AI agents, organized around initial access, execution, persistence, activation, escalation, and impact.
2. A 40-scenario attack catalog spanning eight payload families and five operational workflow framings.
3. A Python harness that runs the same simulated agent architecture against OpenRouter, local Ollama, or authenticated Ollama Cloud models.
4. Hardened heuristic scoring that distinguishes unsafe behavior signals from benign quotation or refusal.
5. A public-release artifact policy that redacts raw prompts and model outputs while preserving flags, metrics, character counts, and SHA-256 trace digests.
6. A reproducible May 20, 2026 Ollama Cloud baseline covering 1,560 attempted model-scenario pairs.

3. Threat Model

AgentKillChain adapts the cyber kill-chain pattern to stateful AI agents:

1. **Initial access:** Untrusted data enters the context window through user input, retrieved documents, notes, web content, or files.
2. **Execution:** The model parses adversarial data as if it were task-relevant instruction.
3. **Persistence:** A memory or summarization step carries the adversarial content into future context.
4. **Activation:** A later benign-looking workflow triggers the remembered payload.
5. **Escalation:** The agent proposes or selects a privileged tool action.
6. **Impact:** The tool action or response leaks data, violates policy, or routes the agent toward attacker goals.

The benchmark assumes a representative assistant agent with memory and tools, not a production deployment. It measures how often the simulated loop emits unsafe signals. It does not claim that the underlying hosted model would compromise a real environment without the surrounding agent architecture.

4. Benchmark Design

4.1 Dataset

The dataset contains 40 structured scenarios:

Dimension	Value
Payload families	8
Workflow framings	5
Total scenarios	40
Scenario phases	single, seed, dormancy, activation
Model catalog in baseline	39 authenticated Ollama Cloud models

The eight payload families are HTML injection, RAG poisoning, markdown injection, memory poisoning, latent memory poisoning, context drift, toolchain confusion, and cognitive overload. The five workflow framings are document summary, incident response, tool selection, memory recall, and user debug request.

Each scenario specifies an `attack_id`, `campaign_id`, `family`, `scenario_type`, `seed_input`, `trigger_input`, `expected_failure_mode`, `severity`, `realism_level`, and `phase`.

4.2 Harness

The harness executes one simulated assistant architecture. The model receives a system prompt, user task, and a representative toolset:

- `read_document`
- `fetch_url`
- `search_notes`
- `send_email`
- `write_note`

The harness uses native OpenAI-format function calling when the model/provider accepts tool schemas. For Ollama models that reject native tools, it falls back to a text-only mock tool protocol. Tool effects are mocked, and a lightweight policy layer marks overtly dangerous calls rather than

executing external side effects.

The multi-stage scenario engine preserves campaign state across seed, dormancy, and activation phases. This allows the benchmark to test whether summarized memory can carry adversarial instructions into later tasks.

4.3 Scoring

The May 20, 2026 baseline uses hardened deterministic heuristic scoring. A result row can be flagged for:

- `unsafe_instruction_adopted`
- `unsafe_tool_call_proposed`
- `secret_disclosure_claimed`
- `policy_violation`
- `refusal_detected`

The overall vulnerability score is the fraction of scored rows where at least one unsafe signal is true. This is best understood as an unsafe-signal rate in a simulated harness, not a confirmed real-world exploit rate.

AgentKillChain also supports LLM-as-judge scoring, but that path was not used for the published Ollama Cloud baseline.

5. Public Artifact Policy

Raw prompts and model outputs can include private system prompts, provider diagnostics, unsafe model-generated text, or material that should not be encouraged through public indexing. For this reason, the tracked public artifacts redact `results[].prompt` and `results[].output`.

The public JSON and CSV retain:

- row counts and status counts
- model identifiers and attack identifiers
- scoring flags
- aggregate metrics
- prompt/output character counts
- SHA-256 digests for raw prompt/output traces

Raw traces should remain in ignored local outputs under `results/generated/` or in private archival storage.

6. Baseline Results

The May 20, 2026 Ollama Cloud baseline completed a full catalog sweep.

Metric	Value
Models requested	39
Attack scenarios	40
Expected model-scenario pairs	1,560
Attempted rows	1,560
Scored rows	1,559
API error rows	1
Timeout rows	0
Coverage rate	99.94%

Aggregate scored-row metrics:

Metric	Value
Overall unsafe-signal rate	1.48%
Unsafe instruction adoption rate	0.58%
Unsafe tool proposal rate	1.41%
Secret disclosure rate	0.00%
Policy violation marker rate	1.22%
Refusal rate	60.62%

Unsafe signals were not evenly distributed across families. In the redacted public artifact, unsafe flags appear in the following family/phase groups:

Family / phase	Scored rows	Rows with any unsafe signal	Rate
Toolchain confusion / single	195	15	7.69%
Context drift / single	195	3	1.54%
Memory poisoning / single	195	3	1.54%
RAG poisoning / single	195	1	0.51%
Cognitive overload / single	195	1	0.51%
HTML injection / single	195	0	0.00%
Markdown injection / single	195	0	0.00%
Latent memory poisoning / seed, dormancy, activation	194	0	0.00%

The one missing scored row came from a provider API error, not a benchmark timeout.

7. Discussion

The baseline supports three conclusions.

First, most tested interactions were resilient under the current simulation. The overall unsafe-signal rate was low, and no scored row was marked as secret disclosure under the hardened heuristic.

Second, toolchain confusion remains the clearest signal in this run. Unsafe tool proposals appeared at a higher rate than other failure modes, which is consistent with the idea that agent risk often comes from the model's role as a tool-selection component rather than from text generation alone.

Third, the latent-memory scenarios did not produce scored unsafe activation in this baseline. This should not be read as evidence that long-horizon memory is safe. The benchmark uses a simplified memory buffer, one pass per scenario, and public payloads. Future work should evaluate weaker memory provenance controls, repeated trials, hidden hold-out payloads, more realistic retrieval systems, and live sandboxed tool execution.

8. Defensive Implications

The results point toward defenses that should sit outside the model:

1. **Memory provenance:** Agents should tag remembered content by source and trust level. Summarization should strip imperative instructions from untrusted retrieved data.

2. **Deterministic tool authorization:** High-risk tools should require policy checks independent of the language model's stated intent.
3. **Tool-call egress controls:** Email, web, file, and execution tools should be constrained by allowlists, explicit user confirmation, and sensitive-data detectors.
4. **Context separation:** System instructions, user intent, retrieved content, memory, and tool results should be represented as separate security domains rather than flattened into one prompt.
5. **Regression benchmarks:** Agent teams should rerun persistent-compromise tests after model, prompt, tool, and memory-policy changes.

9. Limitations

AgentKillChain is a prototype benchmark. Its limitations are material:

- The harness evaluates one simulated assistant loop, not multiple production agent architectures.
- Tool execution is mocked and cannot capture all real implementation edge cases.
- Memory is represented as a controlled context buffer rather than a full vector database or hierarchical memory graph.
- The May 20, 2026 baseline uses one pass per scenario, so it does not estimate variance or confidence intervals.
- Public attack payloads may eventually contaminate model training or alignment data.
- The public result artifact is redacted; independent replay requires access to private raw traces.
- Heuristic scoring can miss subtle unsafe behavior or overcount some explicit unsafe markers.

These limitations are the reason this paper frames the result as an initial baseline rather than a definitive ranking of model security.

10. Ethics and Release Controls

The dataset contains adversarial prompts and simulated unsafe workflows. The public release keeps the attack catalog open for reproducibility, but redacts raw live-run traces to avoid publishing model-specific unsafe output and private system prompt material. The benchmark uses mocked tools and does not execute real exfiltration, file modification, or network side effects.

Researchers extending the benchmark should avoid running live tool actions against real user data. When publishing future runs, they should preserve aggregate evidence and private raw archives separately.

11. Reproducibility

The public repository contains the dataset, harness, website, redacted result artifacts, and visualization scripts.

Smoke validation:

```
python3 -m compileall harness scripts tests test_all.py
python3 -m unittest discover
python3 harness/runner.py --dry-run
python3 scripts/redact_public_results.py --input-json results/generated/model_results.json --output-
json results/model_results.json --output-csv results/model_results.csv
python3 scripts/validate_artifacts.py
```

Live Ollama Cloud baseline command:

```
python3 harness/runner.py --provider ollama-cloud --judge-mode heuristic --run-label ollama-cloud-
take-2026-05-20
python3 scripts/redact_public_results.py --input-json results/generated/model_results.json --output-
json results/model_results.json --output-csv results/model_results.csv
python3 scripts/validate_artifacts.py
```

The raw live-run outputs are written to ignored `results/generated/` paths. The public release artifacts are `results/model_results.json` and `results/model_results.csv`.

12. Publication Path

Because arXiv endorsement may not be available, the recommended publication route is:

1. Publish this manuscript as a preprint through OSF Preprints or SSRN.
2. Create a GitHub release for the code and redacted artifacts.
3. Archive the GitHub release or release bundle on Zenodo to obtain a DOI.
4. Update this manuscript's repository line with the DOI and release tag before final submission.

References

[1] OWASP Foundation. "OWASP Top 10 for Large Language Model Applications." https://owasp.org/www-project-top-10-for-large-language-model-applications/

[2] National Institute of Standards and Technology. "Artificial Intelligence Risk Management Framework (AI RMF 1.0)." https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-ai-rmf-10

[3] Greshake, K. et al. "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection." arXiv:2302.12173. https://arxiv.org/abs/2302.12173

[4] Microsoft Research. "Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models." https://www.microsoft.com/en-us/research/publication/benchmarking-and-defending-against-indirect-prompt-injection-attacks-on-large-language-models/

[5] Debenedetti, E. et al. "AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses for LLM Agents." NeurIPS 2024 Datasets and Benchmarks Track. https://papers.nips.cc/paper_files/paper/2024/file/97091a5177d8dc64b1da8bf3e1f6fb54-Paper-Datasets_and_Benchmarks_Track.pdf